

缓冲区溢出作业

样例1

```
#include <stdio.h>
#include <string.h>
#define MAX_SIZE 256
void safe_function(char *input) {
    char buffer[MAX_SIZE];
    // 使用strncpy防止溢出
    strncpy(buffer, input, sizeof(buffer));
    printf("Received: %s\n", buffer);
}
int main() {
    char input[] = "This is a test string that should not
exceed the maximum size.";
    safe_function(input);
    return 0;
}
```

所在位置： `strncpy(buffer, input, sizeof(buffer));`

形成原因：

strncpy的特性：当源字符串长度 $\geq$ 目标缓冲区大小时，不会自动添加终止符'\0'。

当输入字符串长度 ≥ 256 时，buffer数组将没有终止符，后续printf使用%s格式输出时，会越界读取。

修复方法：

- 安全终止缓冲区：

```
strncpy(buffer, input, sizeof(buffer)-1);  
buffer[sizeof(buffer)-1] = '\0';
```

样例2

```
#include <stdio.h>  
#include <string.h>  
void vulnerable_function() {  
    char buffer[64];  
    printf("Input: ");  
    gets(buffer);  
    printf("You entered: %s\n", buffer);  
}  
  
int main() {  
    setvbuf(stdout, NULL, _IONBF, 0);  
    vulnerable_function();  
    return 0;  
}
```

所在位置： `gets(buffer);`

形成原因：

代码中定义的 buffer 数组只有 64 字节大小。如果用户输入超过 64 个字符，多余的数据会覆盖相邻的内存区域，可能导致程序崩溃、数据损坏，甚至被攻击者利用来执行恶意代码。

修复方法：

fgets 允许指定最大读取长度，从而避免缓冲区溢出。fgets 会从输入中读取数据，并在读取的字符串末尾自动添加 \0。

```

void vulnerable_function() {
    char buffer[64];
    printf("Input: ");
    if (fgets(buffer, sizeof(buffer), stdin) != NULL) {
        printf("You entered: %s\n", buffer);
    } else {
        printf("Error reading input.\n");
    }
}

```

样例4

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int main() {
    char *buffer = (char *)malloc(10);
    if (buffer == NULL) {
        perror("malloc");
        return 1;
    }
    strcpy(buffer, "Hello");
    buffer = (char *)realloc(buffer, 5);
    if (buffer == NULL) {
        perror("realloc");
        return 1;
    }
    strcpy(buffer, "This is a long string that will cause a
heap overflow");
    printf("Buffer: %s\n", buffer);
    free(buffer);
    return 0;
}

```

所在位置:

```
if (buffer == NULL) {  
perror("realloc");  
return 1;  
}
```

```
strcpy(buffer, "This is a long string that will cause a heap  
overflow");
```

形成原因：

1. realloc 缩小内存块：如果realloc 失败并返回 NULL，原始内存块 (buffer) 会丢失，导致内存泄漏
2. strcpy(buffer, "This is a long string that will cause a heap overflow");将一个较长的字符串复制到只有 5 字节的缓冲区中，这会导致堆内存越界写入，破坏堆的结构，可能导致程序崩溃或安全漏洞。

修复方法：

修改点：改为strncpy，对realloc进行修改，防止越界。

全部代码：

```
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
int main() {  
    char *buffer = (char *)malloc(10);  
    if (buffer == NULL) {  
        perror("malloc");  
        return 1;  
    }  
    strncpy(buffer, "Hello", 10);  
    buffer[9] = '\0';  
  
    char *newbuffer = (char *)realloc(buffer, 5);
```

```
if (newbuffer == NULL) {
    perror("realloc");
    free(buffer)
    return 1;
}
buffer=newbuffer;
//手动添加 \0, 确保字符串正确终止
buffer[4] = '\0';

    const char *long_string = "This is a long string that will
cause a heap overflow";

    strncpy(buffer, long_string, 4);
buffer[4] = '\0'; // 确保字符串以 \0 结尾

printf("Buffer: %s\n", buffer);
free(buffer);
return 0;
}
```